

UNIVERSITY EXAMINATION STYLE REVISION PACK

BCS2410 Embedded Programming

Mock Examination Pack

Four simulated exam papers with answer space

Built from the lecture notes, cram sheets, self-tests, worked drills, and syllabus-aligned exam notes in this vault

How this pack is designed

- Each paper is worth **100 marks**.
- Each paper follows the exam structure inferred from your notes: about **40% theory / short closed questions** and **60% practical work in 2 or 3 larger exercises**.
- Across the four papers, all major course areas are covered: foundations, architecture, data representation, C and pointers, ARM Thumb assembly, peripherals and workflow, interrupts and clocks, and FPGA / Edge AI.
- The difficulty is intentionally mixed so you are comfortable whether the real exam is straightforward or demanding.
- A **condensed model solution and marking guide** is included at the end of this PDF.

Exam-use suggestion

- First attempt a paper under timed conditions.
- Mark it only after you have completed the full paper.
- Use the answer boxes as if this were the actual answer script.

Generated in the notes vault:

[/Users/davidwickerhf/Projects/personal/notes/University/BCS2410EmbeddedProgrammingKnowledgeBase/12MockExams/](#)

Contents

Paper 1	1
Paper 2	6
Paper 3	11
Paper 4	16
Condensed Model Solutions and Marking Guide	21

Paper 1

BCS2410 Embedded Programming

MOCK EXAMINATION PAPER

Paper 1: Core Embedded Systems, Data Representation, and ARM Basics

Total marks	100		
Recommended duration	2 hours 10 minutes		
Paper structure	Section A = 40 marks of short theory / closed-ended questions. Section B = 60 marks of practical exercises.		
Candidate instructions	Answer all questions. Show working where appropriate. Write assembly clearly with labels, branch conditions, and register use visible. Where a question requests explanation, concise technical prose is sufficient.		
Candidate name		Student number	
Date		Start time	

Section A: Theory and Short Closed Questions (40 marks)

1. State one key difference between an embedded system and a general-purpose computer. [4 marks]

Answer:

2. Program code on the Discovery board is typically stored in:

- ☐ Random Access Memory (RAM)
- ☐ Flash / ROM-like non-volatile memory
- ☐ Cache only
- ☐ The Arithmetic Logic Unit (ALU)

Answer:

3. What does the Program Counter (PC) hold? [4 marks]

Answer:

4. Name the two main parts of the processor in the datapath-plus-control view. [4 marks]

Answer:

5. In a little-endian system, how is 0x12345678 stored in memory from lowest to highest address? [4 marks]

Answer:

6. What is the main job of the Arithmetic Logic Unit (ALU)? [4 marks]

Answer:

7. Match each flag to its meaning: N, Z, C, V. [4 marks]

Answer:

8. What is the difference between MOV, LDR, and STR? [4 marks]

Answer:

9. Which branch is appropriate for a signed greater-than comparison after CMP: BGT or BHI? Briefly justify. [4 marks]

Answer:

10. What does `*(ptr + 2)` mean when `ptr` is an `int *` pointing at the first element of an array? [4 marks]

Answer:

Section B: Practical Exercises (60 marks)
--

Exercise 1: ARM Thumb loop and branch construction [20 marks]

Write an ARM Thumb function `sum_to_n` with the following behaviour:

- input: `r0 = n`
- output: return `1 + 2 + ... + n` in `r0`
- if `n <= 0`, return 0

Your answer should:

- | | |
|--|------------|
| a) use a loop with labels | [10 marks] |
| b) use the correct signed branch conditions | [5 marks] |
| c) briefly explain why BGT and not BHI is the right exit-test style here | [5 marks] |

Answer:

Exercise 2: Pointers, arrays, and memory layout**[20 marks]**

Consider the following C code:

```
int arr[] = {10, 20, 30, 40};  
int *ptr = arr;  
int a = *(ptr + 2);  
*(ptr + 1) = 99;
```

Answer all parts:

- a) What does `ptr` store? **[4 marks]**
- b) What is the value of `a`? **[4 marks]**
- c) What are the final contents of `arr`? **[4 marks]**
- d) Explain why `ptr + 2` means “two elements later” and not “two bytes later.” **[4 marks]**
- e) Show the byte layout of `0x12345678` in little-endian memory. **[4 marks]**

Answer:

Exercise 3: Memory-mapped I/O and load-modify-store**[20 marks]**

A General-Purpose Input/Output (GPIO) output register address is already stored in `r0`. Write ARM Thumb assembly that:

- a) reads the register **[4 marks]**
- b) sets bit 7 to 1 **[4 marks]**
- c) clears bit 3 to 0 **[4 marks]**
- d) writes the result back **[4 marks]**
- e) briefly explains why ordinary LDR and STR can be used for peripheral registers **[4 marks]**

Answer:

Answer:

Paper 2

BCS2410 Embedded Programming

MOCK EXAMINATION PAPER

Paper 2: Architecture, Flags, Functions, and Assembly Tracing

Total marks	100		
Recommended duration	2 hours 10 minutes		
Paper structure	Section A = 40 marks of short theory / closed-ended questions. Section B = 60 marks of practical exercises.		
Candidate instructions	Answer all questions. Show working where appropriate. Write assembly clearly with labels, branch conditions, and register use visible. Where a question requests explanation, concise technical prose is sufficient.		
Candidate name		Student number	
Date		Start time	

Section A: Theory and Short Closed Questions (40 marks)

1. What is the difference between datapath and control? [4 marks]

Answer:

2. What is the role of the decode stage in the fetch-decode-execute cycle? [4 marks]

Answer:

3. Why are registers faster than ordinary memory access? [4 marks]

Answer:

4. State one difference between Von Neumann and Harvard architecture. [4 marks]

Answer:

5. What is a half adder, and what extra input makes a full adder different? [4 marks]

Answer:

6. What does BL do that B does not? [4 marks]

Answer:

7. Where are the first four 32-bit function arguments usually passed in ARM Cortex-M style calling convention? [4 marks]

Answer:

8. What is the purpose of the Link Register (LR)? [4 marks]

Answer:

9. Why can BGT and BHI give different answers on the same raw bit patterns? [4 marks]

Answer:

10. What is cache for? [4 marks]

Answer:

Section B: Practical Exercises (60 marks)**Exercise 1: Function calls, BL, LR, and tracing****[30 marks]**

Consider the following ARM Thumb code:

ASM_Function:

```
    MOVS r0, #3
    BL   add5
    BL   twice
    BX   LR
```

add5:

```
    ADDS r0, r0, #5
    BX   LR
```

twice:

```
    LSL  r0, r0, #1
    BX   LR
```

Answer all parts:

- a) Trace the value in r0 after each instruction in ASM_Function. **[8 marks]**
- b) What final value is returned in r0? **[4 marks]**
- c) What does each BL instruction do to control flow and to LR? **[8 marks]**
- d) Explain why BX LR returns correctly in this example. **[4 marks]**
- e) If add5 itself needed to call another function, why might it need to preserve LR first? **[6 marks]**

Answer:

Answer:

Exercise 2: Flags, signedness, overflow, and bitwise reasoning**[30 marks]**

Answer all parts.

a) Given:

```
MOV  r0, #-1
MOV  r1, #2
CMP  r0, r1
```

State whether each branch would be taken, and explain why: BGT, BLT, BHI, BLO.

[12 marks]

b) Compute the result of:

```
10110100 AND 00001111
10110100 OR  00001111
10110100 XOR 00001111
```

[6 marks]c) A 32-bit signed integer currently holds 0x7FFFFFFF. What happens when 1 is added? State the new bit pattern and explain the signed interpretation. **[6 marks]**d) Write a short ARM Thumb sequence that sets $r1 = 1$ if $r0$ is odd, else $r1 = 0$. You may use TST or a branch-based solution. **[6 marks]**

Answer:

[illegible]

Answer:

[illegible]

Paper 3

BCS2410 Embedded Programming

MOCK EXAMINATION PAPER

Paper 3: Workflow, Debugging, Peripherals, Clocks, and Integration

Total marks	100		
Recommended duration	2 hours 10 minutes		
Paper structure	Section A = 40 marks of short theory / closed-ended questions. Section B = 60 marks of practical exercises.		
Candidate instructions	Answer all questions. Show working where appropriate. Write assembly clearly with labels, branch conditions, and register use visible. Where a question requests explanation, concise technical prose is sufficient.		
Candidate name		Student number	
Date		Start time	

Section A: Theory and Short Closed Questions (40 marks)

1. What is the difference between build, flash, and debug? [4 marks]

Answer:

2. What is ST-LINK used for? [4 marks]

Answer:

3. Why is the disassembly view useful during debugging? [4 marks]

Answer:

4. What is a breakpoint? [4 marks]

Answer:

5. What is memory-mapped I/O? [4 marks]

Answer:

6. What is the difference between a datasheet and a programming manual? [4 marks]

Answer:

7. What is an interrupt? [4 marks]

Answer:

8. What is an Interrupt Service Routine (ISR)? [4 marks]

Answer:

9. Why do peripherals need clocks? [4 marks]

Answer:

[4 marks]

Exercise 1: Debugging workflow on the Discovery board [20 marks]

Describe a structured debug workflow using course tools and concepts. Your answer should include:

- what to check after build [4 marks]
- what flashing does [4 marks]
- how ST-LINK is involved [4 marks]
- how breakpoints, register views, memory views, and disassembly can help isolate the bug [8 marks]

[illegible]

[20 marks]

```
extern unsigned max3u(unsigned a, unsigned b, unsigned c);
```

Requirements:

- | | |
|--|-----------|
| a) use argument registers correctly | [4 marks] |
| b) use unsigned comparison branches correctly | [8 marks] |
| c) return the result in the correct register | [4 marks] |
| d) briefly explain why signed branches would be wrong here | [4 marks] |

Answer:

[20 marks]

- an LED toggles every 500 ms
- a button press must be detected promptly

-
-
-
-
-

Paper 4

BCS2410 Embedded Programming

MOCK EXAMINATION PAPER

Paper 4: FPGA, Edge AI, and Integrated Embedded System Reasoning

Total marks	100		
Recommended duration	2 hours 10 minutes		
Paper structure	Section A = 40 marks of short theory / closed-ended questions. Section B = 60 marks of practical exercises.		
Candidate instructions	Answer all questions. Show working where appropriate. Write assembly clearly with labels, branch conditions, and register use visible. Where a question requests explanation, concise technical prose is sufficient.		
Candidate name		Student number	
Date		Start time	

Section A: Theory and Short Closed Questions (40 marks)

1. What is a Field-Programmable Gate Array (FPGA)?

[4 marks]

Answer:

2. State one key difference between an FPGA and an Application-Specific Integrated Circuit (ASIC).

[4 marks]

Answer:

3. What is a Lookup Table (LUT) used for in FPGA fabric?

[4 marks]

Answer:

4. What is the role of Block RAM (BRAM)?

[4 marks]

Answer:

5. Why do modern FPGA platforms include Digital Signal Processing (DSP) blocks? [4 marks]

Answer:

6. What is a Deep Processing Unit (DPU) in the course context? [4 marks]

Answer:

7. What is quantization, and why is it useful on edge devices? [4 marks]

Answer:

8. What is a bitstream? [4 marks]

Answer:

9. In a Zynq-style platform, what is the difference between the processing system and programmable logic? [4 marks]

Answer:

10. What does Vitis AI broadly do in the deployment flow? [4 marks]

Answer:

[30 marks]

Answer all parts:

- Answer:*

[illegible]

Answer:

Exercise 2: Mixed embedded-system practical design

[30 marks]

Consider a smart sensor node with:

- a microcontroller
- Flash and Random Access Memory (RAM)
- a timer
- General-Purpose Input/Output (GPIO)
- a status register at address `0x40001000`
- a data register at address `0x40001004`

Bit 0 of the status register becomes 1 when new sensor data is ready.

Answer all parts:

- a) State where program code should live and where live runtime variables should live. **[6 marks]**
- b) Explain why GPIO, timers, and sensor registers count as peripherals rather than the CPU itself. **[6 marks]**
- c) Write ARM Thumb assembly that repeatedly polls the status register until bit 0 is set, then loads the data register value into `r2`. **[12 marks]**
- d) Briefly explain why this is an example of memory-mapped I/O. **[6 marks]**

Answer:

Answer:

Condensed Model Solutions and Marking Guide

Use this section only after attempting a paper. Solutions are intentionally concise and aligned to the vault's course notes rather than to every possible alternative answer.

Paper 1 Solutions

Section A

1. Embedded systems are specialized, hardware-coupled, and resource-constrained; general-purpose systems are meant for broad use.
2. Flash / ROM-like non-volatile memory.
3. The address of the next instruction to execute.
4. Datapath and control.
5. 78 56 34 12.
6. Arithmetic, logic, comparisons, shifts, and related value transformations.
7. N negative, Z zero, C carry / no-borrow style unsigned condition, V signed overflow.
8. MOV moves register / immediate values; LDR loads from memory to a register; STR stores a register to memory.
9. BGT. It is a signed comparison; BHI is for unsigned higher.
10. The value two integer elements after `ptr`; if `ptr` starts at the first element, this is the third element.

Section B

Exercise 1 reference solution

```
sum_to_n:
    CMP    r0, #0
    BLE    done_zero
    MOVS   r1, #1
    MOVS   r2, #0
loop:
    CMP    r1, r0
    BGT    done_sum
    ADD    r2, r2, r1
    ADDS   r1, r1, #1
    B      loop
done_sum:
    MOV    r0, r2
    BX     LR
done_zero:
    MOVS   r0, #0
    BX     LR
```

Marks: correct structure 10, correct signed branches 5, signed-vs-unsigned explanation 5.

Exercise 2 key points

- `ptr` stores the address of `arr[0]`.

- `a = 30`.
- Final array: `{10, 99, 30, 40}`.
- Pointer arithmetic scales by element size.
- Little-endian layout of `0x12345678`: `78 56 34 12`.

Exercise 3 reference solution

```
LDR  r1, [r0]
ORR  r1, r1, #0x80
BIC  r1, r1, #0x08
STR  r1, [r0]
```

BIC clears the selected bit. Full credit also for a correct clear-by-mask alternative. Peripheral registers occupy normal addresses, so LDR and STR work.

Paper 2 Solutions

Section A

1. Datapath moves / transforms values; control issues signals that steer execution.
2. Decode interprets the fetched instruction and generates the internal control actions.
3. Registers are inside the CPU and much closer to the Arithmetic Logic Unit.
4. Von Neumann shares instruction and data memory; Harvard separates them.
5. A half adder adds two bits; a full adder also accepts carry-in.
6. BL branches and stores a return address in LR.
7. In `r0-r3`.
8. It holds the return address after a call-style branch.
9. Signed and unsigned branches interpret the same flag state differently.
10. It reduces average access time by holding frequently used data / instructions close to the CPU.

Section B

Exercise 1

- `MOVS r0, #3` gives `r0 = 3`.
- After `BL add5`, `r0 = 8`.
- After `BL twice`, `r0 = 16`.
- Final returned value is 16.
- Each BL changes control flow and writes the return address into LR.
- `BX LR` works because the callee returns to the address saved by BL.
- If `add5` made another call, that new BL would overwrite LR; it may need `PUSH {lr}` / `POP {lr}` or equivalent preservation.

Exercise 2

- With `r0 = -1` and `r1 = 2`: BGT no, BLT yes, BHI yes, BLO no.
- `10110100 AND 00001111 = 00000100`
- `10110100 OR 00001111 = 10111111`
- `10110100 XOR 00001111 = 10111011`
- `0x7FFFFFFF + 1 = 0x80000000`; under signed 32-bit interpretation this is negative and represents signed overflow.
- One correct odd-test sequence:

```
TST  r0, #1
BEQ  even_case
MOVS r1, #1
B    done
even_case:
    MOVS r1, #0
done:
```

Paper 3 Solutions**Section A**

1. Build creates the binary, flash programs it into the microcontroller, debug runs with inspection and breakpoints.
2. ST-LINK is the host-to-board programming and debugging interface.
3. Disassembly shows generated instructions and addresses, linking source behaviour to machine execution.
4. A breakpoint stops execution at a chosen location.
5. Peripherals occupy addresses in the normal processor address space.
6. Datasheet: device capabilities / characteristics; programming manual: architecture, programming model, register semantics.
7. An interrupt redirects normal execution so an event can be handled promptly.
8. The function / routine that runs in response to an interrupt.
9. Peripherals depend on clock sources to operate correctly.
10. Advantage: simple to understand. Disadvantage: wastes CPU time and can increase latency.

Section B*Exercise 1 expected workflow*

- Confirm successful build and the correct project image.
- Flash the image into device non-volatile memory.
- Use ST-LINK for programming / debug connection.
- Start a debug session, place breakpoints, single-step, inspect variables, inspect registers, inspect memory, compare with disassembly, and identify where behaviour diverges from expectation.

Exercise 2 reference solution

```
.global max3u
max3u:
    CMP    r0, r1
    BHS    keep_r0
    MOV    r0, r1
keep_r0:
    CMP    r0, r2
    BHS    done
    MOV    r0, r2
done:
    BX     LR
```

BHS is unsigned higher-or-same; signed branches would be incorrect because the inputs are explicitly unsigned.

Exercise 3

- Best answer: usually a mixture. Timer interrupt or timer-driven event for periodic LED toggling; interrupt or at least event-based detection for prompt button response; superloop may still exist for background work.
- The timer provides the 500 ms time base.
- High-level sketch: initialize GPIO and timer, enable button interrupt or sampled event, in ISR set a flag or toggle state, main loop handles background tasks.

Paper 4 Solutions**Section A**

1. A reconfigurable integrated circuit whose logic can be programmed after manufacturing.
2. FPGA is reconfigurable after manufacture; ASIC is fixed once fabricated.
3. LUTs implement combinational logic.
4. BRAM provides on-chip storage.
5. DSP blocks accelerate arithmetic-heavy operations.
6. A dedicated accelerator in the FPGA fabric for neural-network inference.
7. Quantization reduces numeric precision, usually reducing model size and compute cost for constrained hardware.
8. The configuration data that programs the FPGA fabric.
9. Processing system is the processor side; programmable logic is the configurable hardware fabric.
10. It supports model preparation, quantization, compilation, and runtime execution on supported targets.

Section B*Exercise 1*

- CPU only: flexible, easiest to program, but less efficient for inference-heavy workloads.

- FPGA + DPU: middle ground; configurable and more efficient for targeted acceleration.
- ASIC: most efficient for one fixed purpose, but inflexible and expensive to design.
- Workflow: prepare model, quantize, compile for target, generate / use platform bitstream and runtime stack, execute with runtime such as VART.
- Quantization reduces precision, often to INT8, to lower cost while keeping acceptable accuracy.
- LUTs implement logic, BRAM stores data and activations, DSP blocks accelerate arithmetic.

Exercise 2

- Program code: Flash. Runtime variables / stack / buffers: Random Access Memory.
- GPIO, timers, and sensor registers are peripherals because they are hardware blocks controlled by the CPU, not the CPU core itself.
- One correct polling solution:

```
    LDR    r0, =0x40001000
poll:
    LDR    r1, [r0]
    TST    r1, #1
    BEQ    poll
    LDR    r1, =0x40001004
    LDR    r2, [r1]
```

- This is memory-mapped I/O because the peripheral registers are accessed through normal addresses with normal load instructions.